

Enterprise Procurement and Inventory Management Platform

REQUIREMENT

Build an Enterprise Procurement and Inventory Management Platform for a fictional retail organisation operating across India, with 150 stores, 12 warehouses, 2,000 suppliers, and 10,000 employees.

The organisation currently manages purchasing, approvals, stock transfers, supplier invoices, and payment tracking through disconnected applications and spreadsheets. This causes duplicate purchases, budget overruns, stock shortages, delayed payments, and limited audit visibility.

The platform must manage the complete lifecycle from identifying a purchasing need through supplier selection, purchase approval, delivery, inventory updates, invoice verification, and payment reconciliation.

1. Users and access

Support these roles:

- Store Requester: creates purchase requests and tracks deliveries for assigned stores.
- Store Manager: approves requests and monitors store budgets.
- Regional Manager: approves higher-value requests across assigned regions.
- Procurement Officer: manages quotations, suppliers, and purchase orders.
- Warehouse Operator: receives goods and processes stock transfers.
- Finance Officer: verifies invoices and schedules payments.
- Finance Director: approves high-value purchases and budget exceptions.
- Supplier: manages quotations, deliveries, and invoices through a supplier portal.
- Auditor: has read-only access to transactions and audit records.
- System Administrator: manages users, organisational structure, and workflow configuration.

Users may hold multiple roles, but must never approve their own requests. Access must be restricted by legal entity, region, store, and warehouse. Suppliers must only access their own records.

2. Organisation and master data

Maintain legal entities, regions, stores, warehouses, cost centres, financial years, product categories, products, units of measure, suppliers, and category budgets.

Products must support SKU, description, category, unit, reorder level, preferred suppliers, and whether batch and expiry tracking are required.

Allow master records to be deactivated without deleting historical transactions.

For the first release, support INR transactions only. Store timestamps consistently and display them in the user's local time zone.

3. *Purchase requisitions*

Users must be able to create and save draft requisitions containing multiple products, quantities, estimated prices, required delivery dates, delivery locations, cost centres, business justifications, and attachments.

Support submission, withdrawal, rejection, revision, cancellation, and tracking.

Before submission, show available stock in the requesting store and nearby warehouses. Suggest a stock transfer when sufficient inventory exists, but allow the requester to explain why purchasing is still necessary.

Each submitted requisition must receive a unique reference number and retain its revision history.

A requisition may be fulfilled through multiple purchase orders, stock transfers, or a combination of both. Track fulfilment at line level to prevent duplicate ordering.

4. *Configurable approval workflows*

Use the following initial rules, based on the total requisition value including entered taxes and delivery charges:

- Up to ■50,000: Store Manager.
- Above ■50,000 and up to ■5,00,000: Store Manager followed by Regional Manager.
- Above ■5,00,000: Store Manager, Regional Manager, and Finance Director in sequence.

Requests exceeding the remaining category budget must receive an additional budget-exception approval from the Finance Director before normal approval begins.

The requester cannot approve any stage of their own request. Route such approvals to a configured alternate approver; if none exists, flag the request for administrator assignment.

Approvers may approve, reject, or return a request for clarification. Rejections and clarification requests require comments.

Support time-limited delegation. Escalate approvals pending for more than two business days using a configurable holiday calendar.

Changes to product, quantity, value, cost centre, or delivery location after submission must create a new revision and restart approval. Previously approved revisions must remain available for audit.

5. Budget controls

Track allocated, reserved, committed, spent, and available budget by cost centre, category, and financial year.

Reserve budget when a requisition receives final approval. Convert the relevant reservation into a commitment when a purchase order is issued. Convert that commitment into spent budget when the corresponding invoice is approved.

Payments must not consume the budget a second time.

Release unused reservations and commitments when requests or purchase orders are cancelled or closed. Approved credit notes must adjust the relevant budget amounts.

For concurrent requests, budget checks and reservations must be atomic so that two approvals cannot consume the same remaining budget.

Budget exceptions must record the approved extra allocation and reason.

6. Supplier onboarding and quotation management

Suppliers must register their company details, contact information, product categories, payment terms, bank details, and supporting documents.

Procurement and Finance must approve onboarding before a supplier can receive a purchase order. Changes to supplier bank details require independent Finance approval.

Support supplier suspension, document-expiry alerts, and performance scores based on delivery timeliness, rejected goods, and invoice discrepancies.

Procurement Officers must be able to request quotations from multiple suppliers and compare price, delivery date, payment terms, and past performance.

Purchases above ■2,00,000 require at least three valid quotations unless an authorised Procurement Manager approves a documented exception. Allow a designated procurement user to hold this approval responsibility.

Choosing a quotation that is not the cheapest requires a recorded justification.

7. Purchase orders and delivery

Generate purchase orders from approved requisition lines and selected quotations. A requisition may produce purchase orders for different suppliers.

A purchase order must include line items, prices, entered taxes, delivery charges, delivery schedules, delivery locations, and payment terms.

If the final order value exceeds the approved amount, require approval of the revised requisition value and additional budget before issuing the order.

Suppliers must be able to accept an order, decline it, or propose delivery changes. Material changes require procurement review.

Support partial deliveries, multiple delivery dates, cancellation of outstanding quantities, and order closure.

Block cancellations for quantities already received; handle those quantities through returns.

8. Goods receipt and inventory

Warehouse and store users must record received, accepted, rejected, and damaged quantities against purchase order lines.

Only accepted quantities increase usable stock. Rejected or damaged goods must remain separately tracked until returned or resolved.

Prevent receipts above the outstanding ordered quantity.

Maintain inventory by product, location, and, where applicable, batch and expiry date. Track on-hand, reserved, available, and in-transit quantities.

Support stock transfers between locations using request, approval, reservation, dispatch, and receipt stages.

Dispatched goods must leave the source location and remain in transit until accepted at the destination. Shortages and damage must create discrepancy records requiring resolution.

Expired stock must not be available for allocation. Inventory adjustments require a reason and manager approval.

Concurrent operations must never produce negative available stock.

9. Invoice verification and payments

Suppliers must submit invoices against purchase orders, including invoice number, date, line items, taxes, total, and an attachment.

Prevent duplicate invoices using supplier identity and a normalised invoice number.

Perform three-way matching between purchase orders, accepted goods receipts, and invoices.

An invoice passes automatic matching only when:

- Invoiced quantities do not exceed accepted quantities that have not already been invoiced.
- Each unit price differs from the purchase order price by no more than 1%.
- Tax and charge totals agree with the order's recorded values and applicable allocation.

Discrepancies must place the invoice on hold for Finance review. Overrides require a reason and an authorised second approver.

Support partial invoices, credit notes, partial payments, and supplier returns. Returns after invoicing must create a credit-note requirement.

Integrate with a payment provider to initiate payments and receive status updates. Track scheduled, processing, successful, failed, and unknown outcomes.

If a payment request times out, reconcile its status before retrying. Duplicate requests and repeated callbacks must never create duplicate payments.

10. Dashboards and reporting

Provide role-specific dashboards showing:

- Pending approvals and overdue tasks.
- Budget allocation, utilisation, and remaining balance.
- Purchase requisition and order status.
- Low-stock and expiring-stock alerts.
- Outstanding deliveries and transfer discrepancies.
- Supplier delivery performance.
- Invoice mismatches, payment failures, and unpaid balances.

Allow filtering by date, legal entity, region, location, supplier, and product category.

Support CSV and Excel exports. Large reports must run in the background and provide a download notification when ready.

11. Notifications and integrations

Provide in-app and email notifications for approvals, rejections, clarifications, purchase orders, delivery delays, invoice holds, and payment outcomes.

Integrate with:

- Corporate single sign-on.
- An ERP system for supplier, cost-centre, and finance synchronisation.
- A payment provider.
- An email provider.
- Object storage for attachments and reports.

Use mock integrations for local demonstration, with clear interfaces for replacing them with real services.

Integration failures must support retries, visible failure status, and authorised manual retry. A temporary email or ERP outage must not erase a completed business transaction.

12. Audit, security, and operational requirements

Maintain an audit trail recording the actor, timestamp, action, entity, previous value, new value, and reason where required.

Restrict supplier bank details and sensitive documents to authorised users. Do not include sensitive information in application logs.

Validate uploads and protect file downloads with access checks.

Support approximately 2,000 concurrent users, 50,000 requisitions per month, and five million inventory movements per year.

Target a response time below three seconds for 95% of normal interactive requests, excluding asynchronous reports and external provider processing.

Include pagination, searchable transaction history, monitoring, structured logging, correlation IDs, health checks, and configurable retention.

13. Required demonstration journeys

The generated prototype and test strategy must cover:

1. A store submits a ■40,000 requisition, the manager approves it, procurement issues an order, goods arrive, and Finance pays a matched invoice.
2. A ■7,00,000 requisition exceeds the remaining budget and passes through budget-exception and sequential purchase approvals.
3. One requisition is split between an inter-warehouse transfer and purchase orders for two suppliers.
4. A supplier delivers only part of an order, some goods are rejected, and payment covers only accepted and invoiced quantities.
5. Two simultaneous requisitions compete for insufficient remaining budget; only the valid reservation succeeds.
6. An invoice exceeds the price tolerance and is held for review.
7. A supplier resubmits an invoice and the duplicate is blocked.
8. A payment times out and later reports success; reconciliation prevents a second payment.
9. An approved requisition is materially edited and requires fresh approval.
10. A supplier attempts to access another supplier's purchase order and is denied.

14. Expected solution

Generate clear requirements, user stories with acceptance criteria, an interactive role-based prototype, architecture, API contracts, an ER diagram, database migrations, backend and frontend code, automated tests, infrastructure configuration, security considerations, a developer checklist, and documentation.

Use Java 21 with Spring Boot for the backend, React with TypeScript for the frontend, PostgreSQL for transactional data, and AWS for the proposed deployment.

Define clear service or module boundaries for procurement, approvals, budgets, suppliers, inventory, invoicing, payments, and notifications. Explain cross-service consistency and recovery where transactions span those boundaries.

Provide seeded demonstration data and local startup instructions. Identify assumptions and unresolved business decisions explicitly, and distinguish implemented features from mocks or future work.

ARCHITECTURE

Architecture and Diagram Summary

- **Frontend Layer:** React with TypeScript for user interfaces and interactions.
- **API Layer:** Spring Boot REST APIs for handling client requests and responses.
- **Application Services Layer:** Java services implementing business logic, separated into modules.

- **Persistence Layer:** PostgreSQL database for storing transactional and master data.
- **Security Layer:** Spring Security for user authentication, authorization, and data protection.
- **External Systems:** Integrations with ERP, payment provider, email service, and object storage.

Code Design

- `com.example.procurement.requisition`: Manages purchase requisitions and workflows.
- `com.example.procurement.approval`: Handles approval processes and rules.
- `com.example.procurement.budget`: Tracks budget allocations and reservations.
- `com.example.procurement.supplier`: Manages supplier onboarding and quotations.
- `com.example.procurement.inventory`: Handles inventory management and stock transfers.
- `com.example.procurement.invoicing`: Manages invoice verification and payments.

Implementation Plan

Phase 1: Setup

Tasks:

- Set up development environment.
- Initialize Spring Boot project.
- Configure PostgreSQL database.

Dependencies: None

Expected Outcome: Development environment ready.

Phase 2: Database

Tasks:

- Design and create database schema.
- Implement entity classes.
- Define repository interfaces.

Dependencies: None

Expected Outcome: Database schema and entity classes ready.

Phase 3: Domain

Tasks:

- Define domain models.
- Implement domain services.
- Create DTOs for API communication.

Dependencies: Database schema

Expected Outcome: Domain models and services implemented.

Phase 4: Business Logic

- Tasks:**
- Implement business logic in services.
 - Define validation rules.
 - Create exception handling.

Dependencies: Domain models

Expected Outcome: Business logic implemented.

Phase 5: APIs

- Tasks:**
- Develop REST APIs.
 - Map DTOs to domain models.
 - Implement API controllers.

Dependencies: Business logic

Expected Outcome: APIs developed.

Phase 6: Testing and Deployment

- Tasks:**
- Write unit and integration tests.
 - Configure security settings.
 - Deploy application to AWS.

Dependencies: APIs

Expected Outcome: Application deployed and tested.

API DESIGN

Endpoints

Method	Path	Purpose	Request Body	Response Body	Success Status	Error Statuses
GET	/users	Retrieve a list of users	N/A	[{"userId":1, "name":"John Doe", "role":"STORE_REQUESTER"}]	200	401, 403, 500

| GET